

Take-Home Assignment

Generic Data Ingestion Service · First round (work on your own) · ~2 days

About this stage

This is the first stage of our technical process. You'll build this on your own, in your own time. Strong submissions are invited to our office for a second round.

The problem

Build a **generic** service that can ingest data from an arbitrary data-source API and store it — **without being written for any one source**.

Given one or more API endpoints as input (for example public e-commerce, ads, or open-data APIs — think of sources such as Amazon or Walmart as the eventual real targets, and we may point it at several endpoints at once), your service should:

- Accept an endpoint (or several) as input / configuration.
- Work out how to pull data from it.
- Download the data.
- Persist it to a database.

The whole point is **generality and extensibility**: adding a new data source should not mean rewriting the application, and it should be straightforward to extend the destination beyond a database later (for example, object storage such as S3). For your demo, connect to any publicly available API you like.

We have **intentionally not given you a checklist of requirements**. Recognizing which concerns matter when ingesting data from real-world APIs, deciding how far to take each within two days, and justifying those calls is a core part of what we are evaluating. Treat the sources as real, external, production APIs and design accordingly.

What to submit

- A **Git repository** containing your code and commit history. Please give our GitHub account hrintentwise **read-only** access to your repository.
- A **hosted endpoint we can call** to trigger and observe ingestion — **strongly preferred**. If hosting isn't feasible for you, include a docker-compose (or equivalent) so we can run it with a single command.
- A **working demo against at least two structurally different public APIs** (for example, different authentication and pagination styles) — enough to show your design is genuinely generic, not tailored to a single source.
- A **short README / design note** covering: how to run it; the public API(s) you used; your architecture and key design decisions; tradeoffs and assumptions; what you would do with more time; and a **short note on how you used AI tools — including one place the AI got something wrong and how you caught it**.

Ground rules

- **Language**: Java or Python (your choice).
- **Time**: about two days.
- **Don't hardcode to a single API** — generality is the point of the exercise.
- **AI tools are expected and encouraged** (Claude Code, Cursor, Copilot, etc.). You own the result and should be able to explain every part of it.

- You may use any publicly available API for your demo.

What we value

- Clear, extensible design and sound engineering judgment over sheer feature count.
- How well you handle the realities of pulling data from live APIs.
- Correctness, and a solution we can actually run and test.
- Clear communication of the decisions and tradeoffs you made.